ĐỒ ÁN TỐT NGHIỆP · LIVESTREAM COMMERCE

LiveOrder

PIPELINE TOÀN DỰ ÁN

Từ comment livestream đến đơn hàng, và AI tư vấn 24/7 — một process, hai lõi thuật toán, một nguồn sự thật.

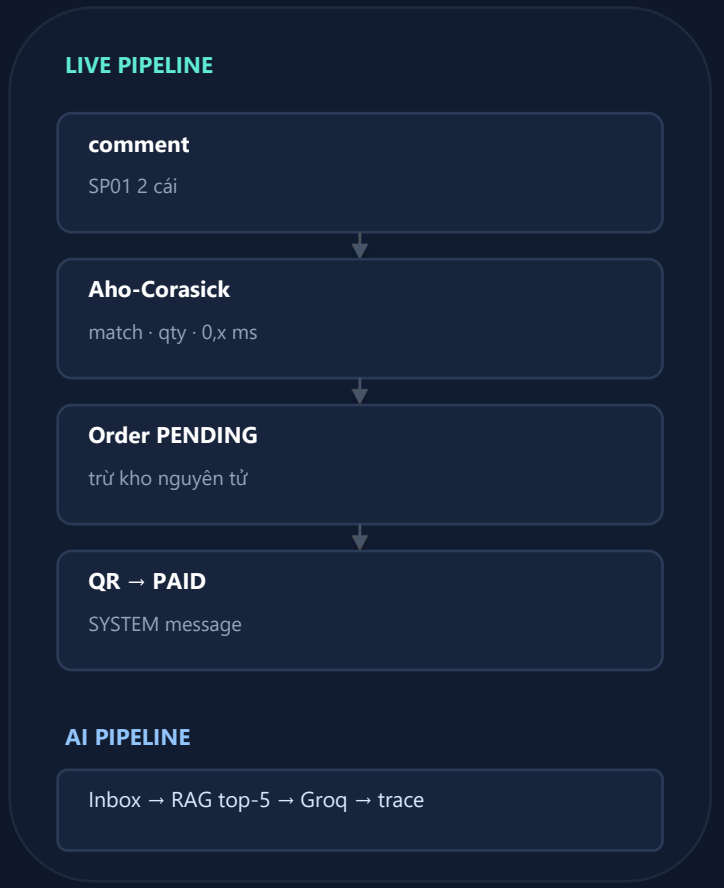
① CỐT LÕI 1

Aho-Corasick — bắt SKU → đơn

② CỐT LÕI 2

RAG — Inbox AI 24/7

Next.js 16 · React 19 · Socket.IO · Prisma 7 / PostgreSQL · e5-small local · Groq



Hai nỗi đau, hai lõi thuật toán

Chốt đơn thủ công

Comment trôi rất nhanh; bỏ sót mã, tính sai số lượng, trừ kho không kịp.

Hỏi ngoài giờ

Shop offline là khách không có ai tư vấn; đơn PENDING dễ bị bỏ quên.

Trả lời sai dữ liệu

Giá, kho, chính sách thay đổi liên tục; AI dễ bịa nếu không có nguồn.

CỐT LÕI 1

Aho-Corasick

Bắt mọi SKU trong MỘT lượt quét; chốt đơn + trừ kho nguyên tử; highlight đúng vị trí; đo thời gian thật bằng performance.now().

trie · fail link · word boundary · atomic transaction

CỐT LÕI 2

RAG

Index KB + sản phẩm theo từng shop; embedding 100% local; top-5 + giá/kho đọc tươi từ DB; Groq trả lời; trace minh bạch từng câu.

e5-small local · cosine top-5 · context DB tươi · fallback FULL_KB

✓ **Không bỏ sót đơn**

comment nào có mã là có phản hồi

✓ **AI 24/7 đúng shop**

kể cả khi shop offline

✓ **Kho không bao giờ âm**

hai khách tranh 1 món → 1 đơn

Một process, hai không gian realtime, một nguồn sự thật

BUYER

Trình duyệt · / · /live/[id] · /inbox · /orders

JWT httpOnly · proxy.ts

Danh tính lấy từ token — không tin payload client

SELLER

Studio · webcam preview · /seller/*

MỘT PROCESS · tsx server.ts

Next.js 16 · App Router

Buyer pages
Seller pages
REST API
auth · products · kb
live · orders
proxy.ts guard

Socket.IO · custom server

/live join_live · send_comment
new_order · viewers
/inbox send_message
conversation_updated
ai_thinking
JWT handshake → socket.data.user

Hai engine lõi

① Aho-Corasick

automaton / phiên · 1 lượt quét · match + vị trí + ms

② RAG + chat flow

index RAM / shop · router 3 nhánh · Groq · trace

PostgreSQL 16 · Docker :5433 · Prisma 7

User · Shop · Product · KbEntry · LiveSession
Conversation · DirectMessage · Order
unique [shopId, sku] · eventId · dedupKey · paymentCode

e5-small · local

multilingual · q8 · 384d
passage: / query:
cache 113 MB

Groq Chat Completions

qwen/qwen3.8-27b
GROQ_MODEL override
timeout 20s

globalThis giữ đúng một embedder + một index cho cả process, dù Next biên dịch code thành nhiều bundle.

Toàn cảnh: live commerce + AI 24/7 chạy song song

LIVE COMMERCE — *một vòng khép kín: mở live → chốt đơn → thanh toán → giao hàng → tổng kết*

1 Seller mở live
Studio webcam
LiveSession = LIVE
socket live_started
→ phiên LIVE

2 Buyer comment
socket /live · JWT
≤200 ký tự
che SĐT trước khi quét
→ text đã che SĐT

3 Aho-Corasick
automaton theo phiên
1 lượt quét
match + vị trí + ms
→ match + 0,x ms

4 Order PENDING
eventId chống trùng
UPDATE stock ≥ qty
+ CREATE Order
→ Order PENDING

5 QR → PAID
QR local + paymentCode
demo pay → PAID
SYSTEM message
→ PAID + SYSTEM

6 Giao hàng
PAID → SHIPPED
Dashboard đọc DB
popup tổng kết live
→ Dashboard

POSTGRESQL — NGUỒN SỰ THẬT DUY NHẤT

Product(giá/kho) · Order(trạng thái) · Conversation(mode/version) · DirectMessage(trace) · LiveSession · KbEntry

AI 24/7 — *song song, không phụ thuộc shop online — nhưng vẫn đúng shop và đúng dữ liệu*

1 Buyer nhắn Inbox
socket /inbox · JWT
lưu BUYER + dedup
phát realtime
→ tin BUYER realtime

2 Router
mode = AI?
câu hỏi về đơn?
3 nhánh rõ ràng
→ 3 nhánh

3 RAG top-5
embed query local
cosine đúng shop
KB + sản phẩm
→ top-5 + score

4 Context + Groq
KB top-5
SP giá/kho tươi
20 tin history
→ prompt

5 Answer + trace
chip nguồn
topK · token · ms
fallback FULL_KB
→ answer + trace

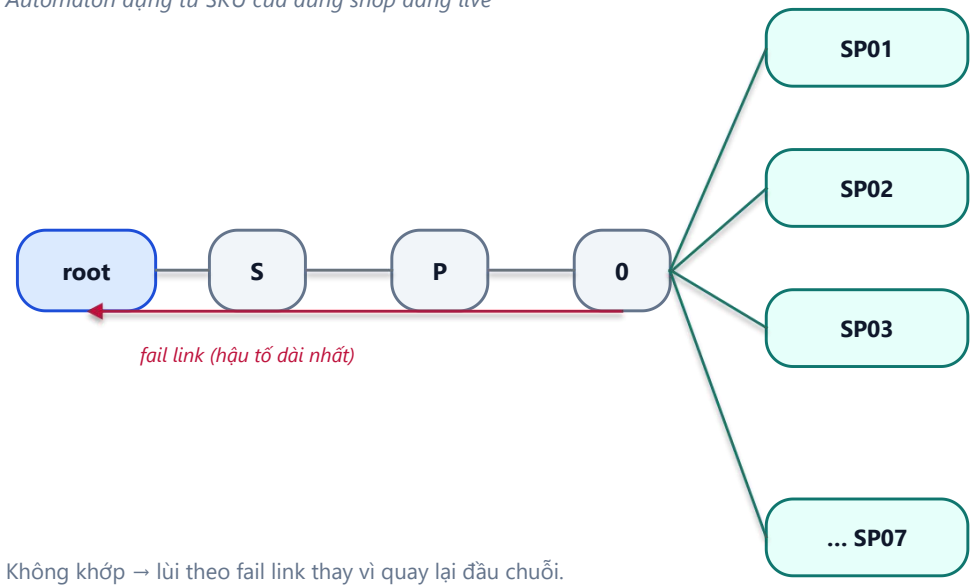
6 Takeover
seller gõ tay
AI dừng ngay
version check
→ AI im lặng

Cùng một process: REST/API + socket /live + socket /inbox. JWT lấy từ handshake; DB là nơi duy nhất quyết định giá, kho và trạng thái.

Aho-Corasick: bắt mọi SKU trong một lượt quét

Trie + fail link (BFS) + output link

Automaton dựng từ SKU của đúng shop đang live



Không khớp → lùi theo fail link thay vì quay lại đầu chuỗi.
Output của nút được lan truyền theo fail link.

Một lượt quét làm gì?

1	**Dựng trie**	mỗi SKU là một đường đi; ký tự chuẩn hoá hoa/thường.
2	**Nối fail link**	BFS: hậu tố dài nhất của tiền tố hiện tại cũng là tiền tố khác.
3	**Quét text**	đi theo cạnh; không có cạnh thì lùi fail — không quay lại đầu.
4	**Chọn match**	mỗi vị trí bắt đầu chỉ giữ match DÀI NHẤT (SP010 thắng SP01).
5	**Word boundary**	ký tự trước/sau không thuộc [A-Z0-9_] → SP01 không khớp trong SP010.
6	**Trả kết quả**	start/end UTF-16 trên chính chuỗi đã che → highlight không lệch.

BUILD
O(Σ \|SKU\|)
dựng một lần / phiên live

QUÉT
O(n + matches)
một lượt trên toàn comment

BỘ NHỚ
1 automaton / phiên
cache theo sessionId

THỜI GIAN
đo thật
performance.now(), không hard-code

Comment → đơn: pipeline nguyên tử, chống trùng

1 Nhận comment

socket /live · JWT handshake
chỉ phiên LIVE · ≤200 ký tự
che SĐT trước khi parse

→ text hợp lệ

2 Aho-Corasick

automaton cache theo phiên
1 lượt quét · match ĐẦU TIÊN
vị trí UTF-16 + ms thật

→ match đầu tiên

3 Chống trùng

eventId =
sha1(session|user|nonce|SKU)
unique trong DB
P2002 → trả đơn cũ

→ eventId

4 Transaction

UPDATE stock WHERE stock ≥ qty
count = 0 → hết hàng
CREATE Order PENDING

→ Order PENDING

5 Phát realtime

comment + matches + ms
new_order → 2 cửa sổ
log Studio · hẹn nhắc tiền

→ 2 cửa sổ

TRANSACTION — hai khách tranh 1 món vẫn đúng 1 người thắng

```
BEGIN;
UPDATE "Product" SET stock = stock - $qty
 WHERE id = $id AND stock >= $qty;   -- count = 0 ⇒ hết hàng
INSERT INTO "Order" (... status='PENDING', unitPrice, total, eventId ...);
COMMIT;
```

Vì sao phải nguyên tử?

Điều kiện stock ≥ qty nằm trong chính câu UPDATE. PostgreSQL row-lock rồi đánh giá lại → không thể cùng trừ.

Tạo đơn

Order PENDING · unitPrice snapshot · paymentCode

Hết hàng

count = 0 · không ghi gì · phát "SPxx đã hết hàng"

Trùng nonce

trả đơn cũ · duplicate: true · không phát new_order lần 2

Bằng chứng: đúng mã, đúng số lượng, đúng thời gian

LIVE — comment thật

```
> lấy SP01 2 cái

match    SP01          [4, 8)
qty      2             (20 ký tự sau mã)
time     0.31 ms       (performance.now)
order    #12 · 300.000đ · PENDING
stock    8 → 6

// comment đã được che SĐT trước khi quét
```

Vị trí [4, 8) trỏ đúng chuỗi phát cho client → highlight không lệch.

Các ca biên đã kiểm thử

INPUT	KẾT QUẢ
SP01 2 cái	✓ match + qty 2
SP010	✗ không khớp SP01
xSP01	✗ sai word boundary
SP01SP02	✗ dính chữ — không mã nào
SP01 100 cái	✓ qty 1 (số 3 chữ số)
0912 345 678	→ 0912***678
2 khách · kho 1	✓ đúng 1 đơn

Đúng shop

SP01 của shop A và shop B là hai sản phẩm khác nhau — automaton chỉ nạp SKU của shop phiên live.

Đúng kho

Trừ kho trong cùng transaction với tạo đơn; điều kiện stock ≥ qty nằm trong UPDATE.

Đúng vị trí

start/end là chỉ số UTF-16 trên chuỗi đã che — client cắt highlight từ chính text nhận được.

RAG: hai pha — nạp một lần, truy vấn mỗi tin nhắn

PHA 1 · NẠP TRI THỨC (LAZY)

mỗi shop một index — chỉ dựng khi cần

1 Nguồn dữ liệu

KbEntry(title, answer) + Product(name, sku)

↓

2 Ghép passage

KB: title + answer
SP: "Sản phẩm {name} (mã {SKU})"

↓

3 Embed local

multilingual-e5-small · q8 · 384d
prefix passage: · mean pooling · normalize

↓

4 Index RAM

Map<shopId> → entries + vectors
vector normalize sẵn → cosine = dot

PHA 2 · TRUY VẤN (MỖI TIN NHẮN)

câu hỏi đi qua router trước khi tới RAG

1 Câu hỏi

buyer gửi qua socket /inbox · JWT

↓

2 Embed query

cùng model · prefix query:

↓

3 Top-5

dot với vector của ĐÚNG shop → sort giảm dần

↓

4 Dựng context

KB top-5 + toàn bộ SP giá/kho tươi
+ SKU exact + live đang mở + 20 tin

↓

5 Groq

qwen3.8-27b · timeout 20s → answer + trace

Giá/kho KHÔNG nằm trong vector — đọc tươi từ DB mỗi lần dựng prompt.

Sửa KB hoặc tên/SKU sản phẩm → invalidateShopIndex(shopId) → lần truy vấn kế tiếp dựng lại.

Pha nạp: index theo shop, lazy, không vector DB

1 Dữ liệu shop

SELECT KbEntry + Product
WHERE shopId = phiên chat

→ KB + Product

2 Kích hoạt

retrieve lần đầu của shop
hoặc sau invalidate

→ buildIndex()

3 Ghép passage

KB: title + answer
SP: tên + mã SKU

→ passages

4 Embed local

e5-small q8 · 384d
mean pooling · normalize

→ 384 chiều

5 Lưu RAM

globalThis Map<shopId>
entries + vectors + meta

→ Map<shopId>

6 Vô hiệu

KB/tên/SKU đổi
invalidateShopIndex(shopId)

→ rebuild lần sau

1 model / process

Singleton globalThis; cache .cache/transformers ~113 MB.
Lần đầu chậm, các lần sau local.

Dot = cosine

Vector đã normalize sẵn; topK mặc định = 5.
Không cần thư viện vector DB.

Cô lập shop

Index riêng theo shopId. SP01 shop A ≠ SP01 shop B.
Không lẫn tri thức.

Không bền

Restart → dựng lại. Đổi giá/kho KHÔNG invalidate vì giá/kho không nằm trong vector.

BẢN GHI INDEX TRONG RAM

```
Map<shopId, ShopIndex>

meta: modelId · revision · builtAt
kbCount · productCount

entries: id · kind(kb|product) · title · sku?
vectors: number[][] · 384 chiều · normalized
```

Pha truy vấn: router 3 nhánh + top-5 + DB tươi

1 Nhận tin

/inbox · JWT
lưu BUYER · dedup nonce
phát realtime

→ BUYER row

2 Router

mode = AI?
câu hỏi về đơn?
3 nhánh rõ ràng

→ nhánh

3 Retrieve

embed query local
dot · top-5 đúng shop
KB + sản phẩm

→ top-5

4 Context

KB top-5
SP giá/kho tươi
SKU exact · live · 20 tin

→ messages

5 Groq

qwen3.8-27b
timeout 20s
1 system + history

→ answer

6 Lưu & phát

AI + trace JSON
socket realtime
chip nguồn + trace

→ UI trace

NHÁNH ROUTER

mode = HUMAN

Seller đã tiếp quản → AI im lặng, không gọi model.

NHÁNH ROUTER

Câu hỏi về đơn

Template đọc DB theo buyerId + shopId. Không gọi LLM, không bịa trạng thái.

NHÁNH GROQ

GroqError

Thiếu key / HTTP lỗi / quá 20s → fallback toàn bộ KB của shop + error trong trace.

NHÁNH TAKE-OVER

version đổi

Seller vừa gõ tay → hủy kết quả AI, không lưu, không phát.

Prompt = 1 system message (vai + quy tắc + NGỮ CẢNH) + 20 tin history + câu hỏi

KB top-5

đọc lại answer theo id

Toàn bộ sản phẩm

giá/kho tươi từ DB

SKU exact

findExactSku nếu câu hỏi nêu mã

Live đang mở

tiêu đề phiên LIVE

20 tin gần nhất

BUYER=user · AI/SELLER=assistant

Vì sao AI không bịa: prompt, trace và 4 chốt chặn

SYSTEM PROMPT — DỰNG TỪ CODE THẬT

```
Bạn là nhân viên tư vấn của shop "…" trên LiveOrder.
Quy tắc:
- Chỉ dùng thông tin trong NGỮ CẢNH; không bịa giá/kho/chính sách.
- Ngoài ngữ cảnh → nói chưa có thông tin, mời để lại câu hỏi.
- Trả lời 1–3 câu, thân thiện, tiếng Việt; xưng "mình", gọi "bạn".

NGỮ CẢNH
- Live đang mở: "…" (nếu có)
- Sản phẩm: SP01 · Áo thun — 150.000đ, còn 8
- Kiến thức top-5: Bảng size: …

HISTORY (20 tin)
BUYER → user · AI/SELLER → assistant · SYSTEM bỏ

USER  1m68 62kg mặc size gì?
```

Trace lưu vào DirectMessage.trace → UI "Vì sao AI trả lời thế này?"

Cô lập shop

Mọi query filter shopId; index riêng. SKU nêu rõ tra chính xác bằng findExactSku — không thay bằng cosine.

Đơn không bịa

Câu hỏi đơn đi template DB theo buyerId + shopId; trạng thái luôn khớp thực tế, không gọi LLM.

Takeover thắng

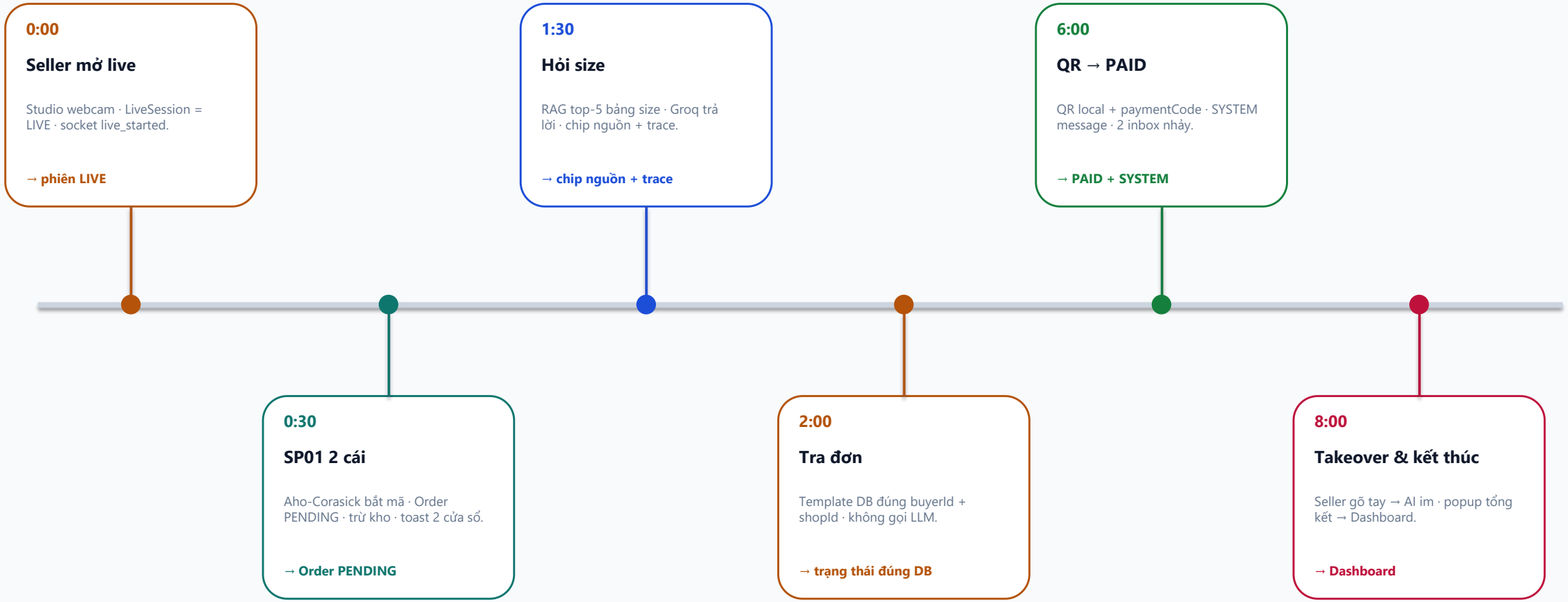
Chốt mode/version lúc dispatch; đọc lại sau khi AI trả lời. Lệch → hủy im lặng, không lưu, không phát.

Trace minh bạch

model · topK + score · products · tokens · elapsedMs · fallback/error. Xem được ngay trên UI.

e5-small 384d · topK 5 · cosine · history 20 · qwen3.8-27b · timeout 20s · fallback FULL_KB

Hai lỗi gặp nhau trong một phiên demo 10 phút



Mỗi mốc đều để lại bằng chứng: log ms ở Studio, chip nguồn + trace ở Inbox, trạng thái đơn trong DB và Dashboard.

Điều gì làm nên pipeline này

① Aho-Corasick

comment → SKU → đơn

1 lượt quét	cho mỗi comment
O(n + matches)	không quét lại theo từng SKU
0,x ms	đo bằng performance.now()
Kho nguyên tử	UPDATE … WHERE stock >= qty

② RAG

tri thức shop → câu trả lời

top-5	cosine trên vector đã normalize
384 chiều	e5-small, embedding 100% local
20 giây	timeout Groq + fallback toàn KB
Trace	model · nguồn · token · thời gian

1 Aho-Corasick

biến comment thành đơn — nhanh, đúng, không oversell.

2 RAG

biến dữ liệu shop thành câu trả lời — đúng shop, đúng giá/kho, có dẫn nguồn.

3 Niềm tin

takeover + trace + fallback: người thật luôn thắng, AI không bịa.

Next.js 16 · React 19 · Socket.IO · Prisma 7 / PostgreSQL · Transformers.js e5-small · Groq

Cảm ơn hội đồng — Q&A